

CODE LAYOUT, READABILITY AND REUSABILITY

FITBUDDY AI FITNESS PLAN GENERATOR – GEMINI MODELS

Project Information	Details
Date	30 September 2026
Team ID	Not provided
Project Name	Fitbuddy AI Fitness Plan Generator – Gemini Models
Maximum Marks	5 Marks
Team Leader	Abisreeya S
Team Members	Sahana K; Sripriya C; Rajakaleeshwari N

Purpose

This document reviews the planned code quality for Fitbuddy, an AI-powered fitness-plan generator that uses Gemini models. It follows the supplied Code-Layout, Readability and Reusability template. Since source code was not provided, checklist statuses and ratings below are proposed sample assessments for the project team to verify against the implemented code.

1. Code Layout Checklist

S.No	Code Quality Parameter	Description / Expected Practice	Followed	Remarks
1	Consistent Indentation	Use consistent spacing throughout the code.	Partial	Confirm using the project formatter.
2	Proper File Structure	Organize UI, API/model integration, fitness-plan logic, configuration, and tests into clear folders.	Partial	Adopt a documented folder structure.
3	Meaningful Variable Names	Use names such as <code>user_profile</code> , <code>fitness_goal</code> , and <code>workout_plan</code> .	Partial	Review names during code review.
4	Function / Method Names	Use descriptive names such as <code>generate_fitness_plan()</code> and <code>validate_user_profile()</code> .	Partial	Keep functions focused on one task.
5	Code Comments	Document non-obvious logic, Gemini prompts, validation rules, and assumptions.	Partial	Add comments where they clarify why.
6	Modular Design	Separate profile validation, Gemini requests, plan formatting, and safety checks into reusable modules.	Partial	Avoid putting all logic in one file.
7	No Redundant Code	Reuse shared validation and formatting helpers instead of duplicating code.	Partial	Check for duplicate or unused functions.
8	Error Handling	Handle invalid input, network/API failures, empty model responses, and timeouts with clear messages.	Partial	Test failure cases and protect API keys.

2. Reusable Components / Modules

S.No	Component / Module Name	Language / Technology	Where Reused	Reusability Level
1	User Profile Validator	Project language	Profile form and plan-generation flow	High
2	Gemini API Service	Gemini API SDK	Fitness-plan generation requests	High

3	Fitness Goal & Activity Mapper	Project language	Workout and activity recommendations	Medium
4	Plan Formatter	Project language	Daily and weekly plan display/export	High
5	Error & Safety Handler	Project language	Validation, API failures, and safety messages	High

3. Overall Code Quality Assessment

Aspect	Rating (1–5)	Comments
Code Layout & Structure	3 / 5 (provisional)	Suggested structure is defined; verify the actual repository.
Readability	3 / 5 (provisional)	Use clear naming, consistent formatting, and focused functions.
Reusability	3 / 5 (provisional)	Proposed shared modules can be reused across plan-generation features.
Documentation / Comments	3 / 5 (provisional)	Document setup, environment variables, Gemini integration, and assumptions.
Overall Score	3 / 5 (provisional)	Sample planning score only; update after reviewing implemented code.

4. Recommended Review Actions

- Apply a formatter and linting rules consistently.
- Keep Gemini API keys in environment variables or a secrets manager; do not hard-code credentials.
- Validate user inputs and handle API timeouts, quota errors, and malformed model responses.
- Add unit tests for profile validation, plan formatting, and error-handling paths.
- Include clear limitations for generated fitness suggestions and recommend qualified professional advice for medical concerns.
- Replace provisional checklist statuses and ratings with evidence from the completed codebase before submission.

Prepared for the Fitbuddy project team